

Basics of Object Oriented Programming

1. Object-Oriented Programming

- **Object-Oriented Programming (OOP)** is a programming technique in which programs are written on the basis of objects.
- An **object** is a collection of data and functions.
- Significant object-oriented languages include:
 - Python
 - Java
 - Go
 - Ruby
 - C++

1.1. Features of OOP

Following are some features of OOP:

1. **Objects**

OOP provides the facility of programming based on objects. Objects is an entity that consists of data and functions.

2. **Classes**

Classes are designed for creating objects. All properties and functions of an object are specified in classes.

3. **Real-world**

As in real world, things have properties and working capabilities. Similarly, objects have data and functions. Data represents properties and functions represent working of objects.

4. **Reusability**

OOP provides ways of reusing the data and code. **Inheritance** is a technique that allows a programmer to use the code of existing program to create new programs.

1.1. Features of OOP (continue...)

5. **Polymorphism**

Polymorphism is an ability of an object to behave in multiple ways.

2. Objects

An **object** represents an entity in the real world such as a person, thing or concept etc. An object is identified by its name. An object consists of the following two things:

- **Properties:** Properties are the characteristics of an object.
- **Functions:** Functions are the actions that can be performed by an object.

Examples- some examples of objects of different types are as follows:

- **Physical Objects** (Car, Bus, Truck etc.)
- **Elements of Computer-User Environment** (Windows, Menus, Graphic Objects, Mouse and Keyboard)
- **Collection of data** (A personnel file, A table of marks related to examination etc.).
- **User-Defined Data Types** (Time, Angles etc.).

2.1. Properties of Object

The characteristics of an object are known as its **properties** or **attributes**. Each object has its own properties. These properties can be used to describe the object. For example, the properties of an object Person can be as follows:

- Name
- Age
- weight

2.2. Functions of Object

An object can perform different tasks and actions. The actions that can be performed by an object are known as **functions** or **methods**. For example, the object **Car** can perform the following functions:

- Start
- Stop
- Accelerate
- Reverse

3. Classes

- A collection of objects with same properties and functions is known as **class**. A class is used to define the properties/characteristics of the objects. It is used as a model for creating different objects of same type.
- For example, a class person can be used to define the characteristics and functions of a person. It can be used to create many objects of type **Person** such as Ali, Usman, Abdullah etc. All objects of **Person** class will have same characteristics and functions.
- Each object of a class is known as an **instance** of its class. For example, Ali, Usman and Abdullah are three instances of a class **Person**.

3.1. Declaring a Class

A class is declared in the same way as a structure is declared. The keyword **class** is used to declare a class. A class declaration specifies the variables and functions that are common to all objects of that class. The variables declared in a class are known as **member variables** or **data members**. The functions declared in a class are called **member functions**.

Syntax- The syntax of declaring a class is as follows:

```
class identifier
{
    body of the class
};
```

Class: Keyword to declare a class

Identifier: Name of the class

The class declaration always ends with a semicolon.

3.1. Declaring a Class (continue...)

Example:

```
class Test
{
    int n;
    char c;
    float x;
};
```

The above example class only contains data members in it. A class may contain both data members and member functions.

3.2. Access Specifiers

The commands that are used to specify the access level of class members are known as access specifiers. Two most important access specifiers are as follows:

1. The 'private' Access Specifier

The **private** access specifier is used to restrict the use of class member within the class. Any member of the class declared with **private** access specifier can only be accessed within the class. It cannot be accessed from outside the class. *It is also the default access specifier.*

The data members are normally declared with **private** access specifier. It is because the data of an object is more sensitive. The **private** access specifier is used to protect the data member from direct access from outside the class. These data members can only be used by the functions declared within the class.

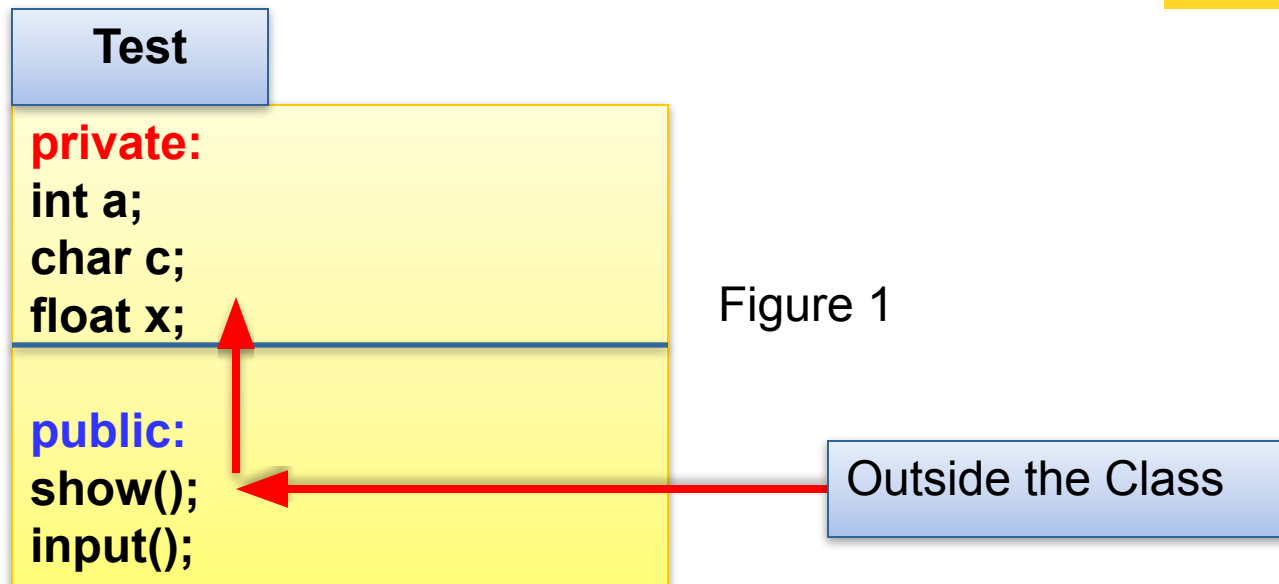
3.2. Access Specifiers (continue...)

2. The 'public' Access Specifier

The **public** access specifier is used to allow the user to access a class member within the class as well as outside the class. Any member of the class declared with public access specifier can be accessed from anywhere in the program.

The member functions are normally declared with **public** access specifier. It is because the users access functions of an object from outside the class.

3.2. Access Specifiers (continue...)



The above figure shows that data members **a**, **c** and **x** of class **Test** cannot be accessed from outside the class because they are declared with **private** access specifier. The member functions **show()** and **input()** are accessible from outside the class because they are declared with public access specifier. The data members are accessed and manipulated indirectly through member functions.

4. Creating Objects

- A class is simply a model or prototype for creating objects. It is like a new data type that contains both data and functions. An object is created in the same way as other variables are created. When an object of a class is created, the space for all data members defined in the class is allocated in the memory according to their types. An object is also known as **instance**. The process of creating an object is called **instantiation**.

Syntax- The syntax of creating an object is as follows:

```
class_name object_name;
```

Example

```
Test obj;
```

The above example declares an object obj of type **Test**. The object contains all data members that are defined in class **Test**.

4.1. Executing Member Functions

An object of a particular class contains all data members as well as member functions defined in that class. The data members contain the value related to the object. The member functions are used to access/manipulate data members. The member functions can be executed only after creating an object.

Syntax- The syntax of executing member functions is as follows:

object_name.member function();

Example

Test obj;

obj.input();

The object name and member function are separated by **dot operator**.

4.1. Executing Member Functions (continue...)

Program 1: Write a program that declares a class with one integer data member and two member functions in() and out() to input and output data in data member.

```
#include<iostream>
using namespace std;
class Test
{
private:
int n;
public:
void in()
{
    cout<<"Enter a number: ";
    cin>>n;
}
void out()
{
    cout<<"The value of n = "<<n;
}
};
```


4.1. Executing Member Functions (continue...)

```
int main()
{
    Test obj;
    obj.in();
    obj.out();
    return 0;
}
```

Output

```
Enter a number: 10
The value of n = 10
-----
```

4.1. Executing Member Functions (continue...)

Program 2: Write a class Marks with three data members to store three marks. Write three member functions **in()** to input marks, **sum()** to calculate and return the sum, **avg()** to calculate and return the average marks.

```
#include<iostream>
using namespace std;
class Marks
{
private:
int a, b, c;
public:
void in()
{
    cout<<"Enter three marks: ";
    cin>>a>>b>>c;
}
int sum()
{
    return a+b+c;
}
```

4.1. Executing Member Functions (continue...)

```
float avg()
{
    return(a+b+c)/3.0;
}

int main()
{
    Marks m;
    int s;
    float a;
    m.in();
    s = m.sum();
    a = m.avg();
    cout<<"Sum = "<<s<<endl;
    cout<<"Average = "<<a;
    return 0;
}
```

Output

```
Enter three marks: 50
40
50
Sum = 140
Average = 46.6667
-----
```

4.2. Defining Member Functions outside class

The member function of a class can also be defined outside the class. The declaration of member function is specified within the class and function definition is specified outside the class. The scope **resolution operator ::** is used in function definition if the function is declared within the class.

Syntax- The syntax of defining member function outside the class is as follows:

```
Return_type class_name::function_name(parameters)
{
    function body;
}
```

4.2. Defining Member Functions outside class (continue...)

Program 3: Write a program that declares a class with one integer data member and two member functions in() and out() to input and output data in data member.

```
#include<iostream>
using namespace std;
class Test
{
private:
int n;
public:
void in();
void out();
};
void Test::in()
{
    cout<<"Enter a number: ";
    cin>>n;
}
```

4.2. Defining Member Functions outside class (continue...)

```
void Test::out()
{
    cout<<"The value of n = "<<n;
}

int main()
{
    Test obj;
    obj.in();
    obj.out();
    return 0;
}
```

Output

```
Enter a number: 10
The value of n = 10
-----
```

5. Constructors

- **Constructor** is a type of member function that is automatically executed when an object of that class is created. The constructor has no return type and has same name as that of the class name. The constructor can work as a normal function but *it cannot return any value*. It is normally defined in classes to initialize data member.

Syntax- The syntax of declaring constructor is as follows:

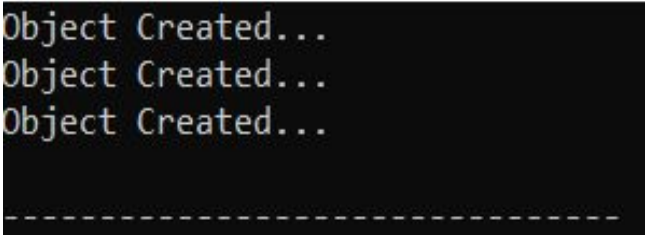
```
class_name()  
{  
    constructor body;  
}
```

5. Constructors (continue...)

Program 4: Write a class that displays a simple message on the screen whenever an object of that class is created.

```
#include<iostream>
using namespace std;
class Hello
{
private:
int n;
public:
    Hello() // constructor can't even take void as return type
    {
        cout<<"Object Created..."<<endl;
    }
};
int main()
{
    Hello x, y, z;
    return 0;
}
```

Output



```
Object Created...
Object Created...
Object Created...
-----
```

*This program declares a constructor that displays a message on the screen. The program creates three objects in **main()** function. The constructor is executed each time an object of the class is created in the memory.*

5.1. Passing Parameters to Constructors

The method of passing parameters to a constructor is same as passing parameters to normal functions. The only difference is that the parameters are passed to the constructor when the object is created. The parameters are written in parenthesis along with the object name at the time when object is created.

Syntax- The syntax of passing parameters to constructor is as follows:

class_name **object_name** (**parameters**);

5.1. Passing Parameters to Constructors (continue...)

Program 5:

```
#include<iostream>
using namespace std;
class Student
{
private:
int marks;
char grade;
public:
    Student(int m, char g)
    {
        marks = m;
        grade = g;
    }
    void show()
    {
        cout<<"Marks = "<<marks<<endl;
        cout<<"Grade = "<<grade<<endl;
    }
};
```

Output

```
Record of Student 1:
Marks = 730
Grade = A
Record of Student 1:
Marks = 621
Grade = B
-----
```

5.1. Passing Parameters to Constructors (continue...)

Program 5: (continue...)

```
int main()
{
    Student s1(730, 'A');
    Student s2(621, 'B');
    cout<<"Record of Student 1:"<<endl;
    s1.show();
    cout<<"Record of Student 1:"<<endl;
    s2.show();
    return 0;
}
```

5.2. Constructor Overloading

The process of declaring multiple constructors with same name but with different parameters is known as **constructor overloading**. The constructor with same name must differ in one of the following ways:

- Number of parameters
- Type of parameters
- Sequence of parameters

5.2. Constructor Overloading (continue...)

Program 6:

```
#include<iostream>
using namespace std;
class Over
{
private:
int num;
char ch;
public:
Over()
{
    num = 0;
    ch = 'x';
}
Over(int n, char c)
{
    num = n;
    ch = c;
}
```

5.2. Constructor Overloading (continue...)

Program 6: (continue...)

```
void show()
{
    cout<<"num = "<<num<<endl;
    cout<<"ch = "<<ch<<endl;
}
};
int main()
{
    Over first;
    Over second(100, 'p');
    cout<<"The contents of first: "<<endl;
    first.show();
    cout<<"The contents of second: "<<endl;
    second.show();
    return 0;
}
```

Output

```
The contents of first:
num = 0
ch = x
The contents of second:
num = 100
ch = p
-----
```

5.3. Default Copy Constructor

A type of constructor that is used to initialize an object with another object of the same type is known as **default copy constructor**. Its name is “default copy constructor” because it is available by default in all classes.

Syntax- The syntax of using default copy constructor is as follows:

class_name **object_name** (**parameters**);

OR

class_name **object_name** = **parameter**;

5.3. Default Copy Constructor (continue...)

Program 7:

```
#include<iostream>
using namespace std;
class Book
{
private:
int pg, pr;
char title[10];
public:
void get()
{
    cout<<"Enter Title: ";
    cin>>title;
    cout<<"Enter Pages: ";
    cin>>pg;
    cout<<"Enter Price: ";
    cin>>pr;
}
```

Output

```
Enter Title: OOP
Enter Pages: 637
Enter Price: 200

The detail of b1:
Title: OOP
Pages: 637
Price: 200

The detail of b2:
Title: OOP
Pages: 637
Price: 200

The detail of b3:
Title: OOP
Pages: 637
Price: 200
-----
```


5.3. Default Copy Constructor (continue...)

Program 7: (continue...)

```
void show()
{
    cout<<"Title: "<<title<<endl;
    cout<<"Pages: "<<pg<<endl;
    cout<<"Price: "<<pr<<endl;
}
};
int main()
{
    Book b1;
    b1.get();
    Book b2 = b1; //The values of data members of object "b1" are copied to data members of new object "b2".
    Book b3 = b1; //The values of data members of object "b1" are copied to data members of new object "b3".
    cout<<"\n The detail of b1:"<<endl;
    b1.show();
    cout<<"\n The detail of b2:"<<endl;
    b2.show();
    cout<<"\n The detail of b3:"<<endl;
    b3.show();
    return 0;
}
```

Type of b1, b2 and b3 must be same i.e., b1, b2 and b3 must all be the objects of same class Book.

6. Destructors

- **Destructor** is a type of member function that is automatically executed when an object of that class is destroyed. The destructor has no return type and has same name as that of the class name. The destructor *cannot return any value*. It also cannot accept any parameters. The constructor name is preceded by tilde sign ~ .

Syntax- The syntax of declaring destructor is as follows:

```
~ name()  
{  
    destructor body;  
}
```

6. Destructors (continue...)

Program 8:

```
#include<iostream>
using namespace std;
class Test
{
private:
int n;
public:
Test()
{
cout<<"Object created..."<<endl;
}
~Test()
{
cout<<"Object destroyed..."<<endl;
}
};
int main()
{
Test a, b;
return 0;
}
```

Output

```
Object created...
Object created...
Object destroyed...
Object destroyed...
-----
```

7. Objects as Function Parameters

Objects can also be passed as parameters to member functions. The method of passing objects to a function as parameters is same as passing other simple variables.

7. Objects as Function Parameters (continue...)

Program 9:

```
#include<iostream>
using namespace std;
class Travel
{
private:
int km, hr;
public:
Travel()
{
km=hr=0;
}
void get()
{
cout<<"Enter kilometers traveled:";
cin>>km;
cout<<"Enter hours traveled: ";
cin>>hr;
}
```

Output

```
Enter kilometers traveled:100
Enter hours traveled: 2
You traveled 100 kilometers in 2 Hours
Enter kilometers traveled:300
Enter hours traveled: 3
You traveled 300 kilometers in 3 Hours
Total traveling is 400 kilometers in 5 hours
-----
```

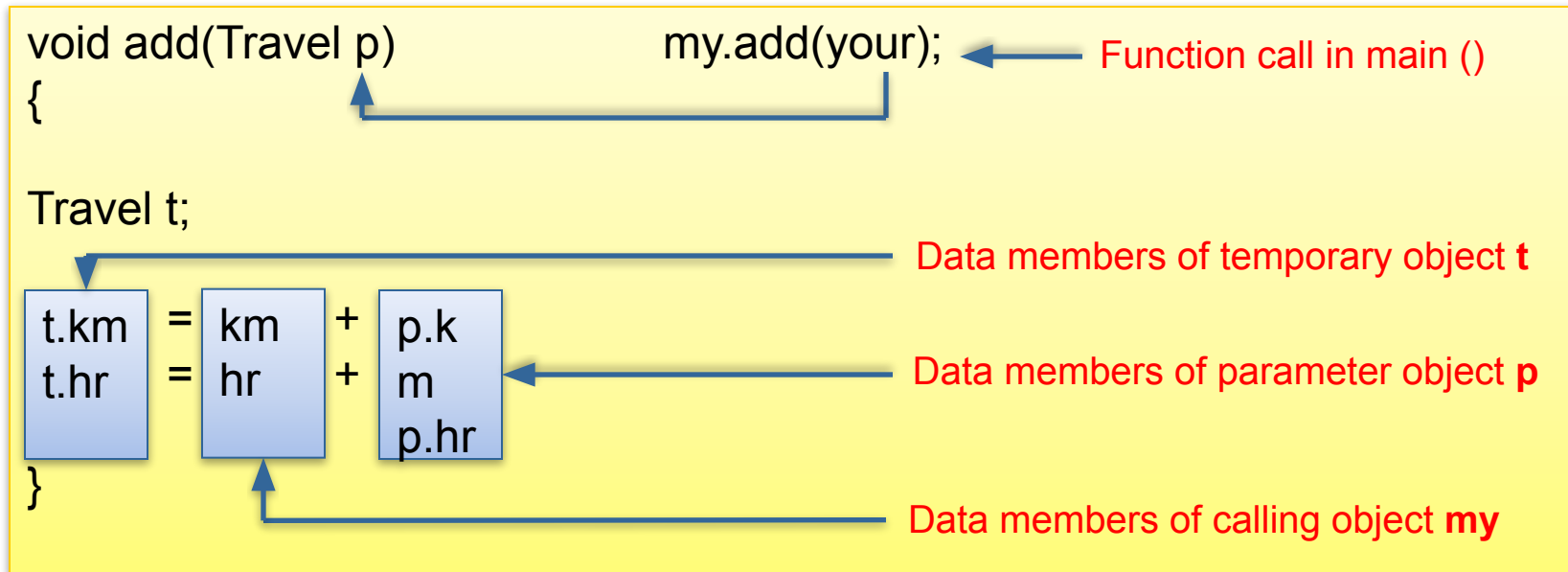
7. Objects as Function Parameters (continue...)

Program 9: (continue...)

```
void show()
{
    cout<<"You traveled "<<km<<" kilometers "<<" in "<<hr<<" Hours"<<endl;
}
void add(Travel p)
{
    Travel t;
    t.km = km + p.km;
    t.hr = hr + p.hr;
    cout<<"Total traveling is "<<t.km<<" kilometers in "<<t.hr<<" hours "<<endl;
}
};
int main()
{
    Travel my, your;
    my.get();
    my.show();
    your.get();
    your.show();
    my.add(your);
    return 0;
}
```

7. Objects as Function Parameters (continue...)

How Program 9 works?



In the above figure, **t.km** and **t.hr** are the data members of the temporary object **t**. **p.km** and **p.kr** are the data members of the parameter object **p**. The data members without dot operator are the data members of calling object **my**. It means that any data member that is not preceded by object name in a member function represents the data member of **calling object**.

7.1. Returning Objects from Member Functions

The method of returning an object from member function is same as returning a simple variable. If a member function returns an object, its return type should be the same as the type of object to be returned.

7.1. Returning Objects from Member Functions (continue...)

Program 10:

```
#include<iostream>
using namespace std;
class Travel
{
private:
int km, hr;
public:
Travel()
{
km=hr=0;
}
void get()
{
cout<<"Enter kilometers traveled:";
cin>>km;
cout<<"Enter hours traveled: ";
cin>>hr;
}
```

Output

```
Enter kilometers traveled:100
Enter hours traveled: 2
You traveled 100 kilometers in 2 Hours
Enter kilometers traveled:300
Enter hours traveled: 3
You traveled 300 kilometers in 3 Hours
Total traveling is 400 kilometers in 5 hours
-----
```

7.1. Returning Objects from Member Functions (continue...)

Program 10: (continue...)

```
void show()
{
    cout<<"You traveled "<<km<<" kilometers "<<" in "<<hr<<" Hours"<<endl;
}
void add(Travel p)
{
    Travel t;
    t.km = km + p.km;
    t.hr = hr + p.hr;
    return t;
}
};
int main()
{
    Travel my, your, r;
    my.get();
    my.show();
    your.get();
    your.show();
    r = my.add(your);
    r.show();
    return 0; }
```

References

1. *“C++ Programming: From Problem Analysis to Program Design”, 6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *“Object Oriented Programming in C++,” 4/ed by Robert Lafore (2001) .*
3. *“C++ How to Program,” 5/ed by Deitel & Deitel (2005).*
4. *“Program with C++ Object Oriented Programming Aikman Series”, by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*